PLAN

Prototype — MIG 000001-1 Mainframe → Target System pipeline on GCP

> **This is the original build plan, written before implementation started — kept as a record of intent, not as a description of the current system.** All 13 phases were completed. Where the plan and the code disagree, the code is right; the two known divergences are worth naming here so nobody follows the plan into them:
>
> - **Flex Templates (§ item 2, item 7).** Built and published, but the DAG does *not* launch them — a Flex Template launch stages rather than runs the job. Every task is a `KubernetesPodOperator` pod instead.
> - **Dataform operators (§ item 6, item 7).** Not used: the Dataform repository is unlinked, so `git_commitish` cannot resolve. `dataform_run` is a pod running `dataform compile --json` plus the executor.
> - **The BigQuery writers (§ item 3).** The plan describes `NativeBigQueryWriter` and `EmulatorBigQueryWriter` with FILE_LOADS as "not written yet". That was true until 2026-08-18 and is no longer: the classes are `InsertAllBigQueryWriter` and `FileLoadsBigQueryWriter`, the latter staging NDJSON in GCS and issuing a load job, and both pipelines resolve their writer through `sinks.bigquery_writer(cfg)`.
>
> For current state see `../README.md` and `runbook-gcp.md`.
>
> **Superseded 2026-08-21 — three simplifications.** The engine model this plan was written against has since been simplified; the items below that describe the old model are now historical, not targets. Specifically superseded by `PLAN-CHANGES-21082026.md`: the **four-disposition** balancing equation (`src_read = written + excluded + rejected + deduplicated`) → two dispositions (`src_read = written + rejected`); the **run-id/window + delta** "must prove" items and the `make run-delta` build phase → one full snapshot per run, scoped by `run_id` only; the **dual `.DAT` + JSON** TDS section (per-field `fmt`) → homogeneous definitions (`.DAT` *or* JSON, never mixed); and the **"10 acceptance criteria" / "seeded excluded count"** verification items → 8 criteria. The phase-by-phase build sequence below is left as the record of intent it always was.

Context

`architecture diagram` (MIG 000001-1) describes a three-lane migration pipeline: Mainframe Db → **Extraction** → **Transformation & Reconciliation** on GCP → **Load** into Target System. The working directory is empty apart from the spec image; this builds the prototype from scratch.

Decisions taken

Decision	Choice
Architecture	`architecture diagram` as drawn — Dataflow ×3, BigQuery ×2 datasets, Dataform, Composer, recon services
Target env	Local Docker Compose stack now + full Terraform for GCP
Chain scope	Full E → T+R → L; Other Team's Extractor/Loader mocked, Our Team's T+R real
Language	Beam pipelines + Airflow DAGs in **Python**; Extractor/Loader/Recon/Target-System-mock in **Java**
Source data	Fixed-width COBOL copybook **and** delimited CSV, switchable
Output sink	**Both** behind one adapter: Kafka 200-batches *and* GCS JSON → Target System mock
Must prove	Balancing equation + two doors; TDS dual `.DAT` +JSON; zero-diff `project2` ; run id/window + delta

---

Component map — every box in architecture diagram gets a deliverable

architecture diagram box	Deliverable	Stack
Mainframe	`harness/` synthetic Db2 generator	Python
Extractor App	`apps/extractor-app` → `.DAT/.CHS/.ERR/.RPT/.FLG` , archive→gzip→PGP	Java
File Storage (landing)	`fake-gcs-server` locally / GCS bucket in TF	—
Dataflow File Processor / Data Loader	`pipelines/file_processor` — decrypt, decompress, verify `.CHS` , TDS parse, parse/filter/dedup → BQ	Beam Python
BigQuery Extraction DataSet	`bq_extraction` dataset	BQ emulator / BQ
Dataform (SQL Transformation)	`dataform/` SQLX models, Extraction → Transformation	SQLX
BigQuery Transformation DataSet	`bq_transformation` dataset	BQ emulator / BQ
Dataflow Data Enrichment	`pipelines/data_enrichment` — reference-data joins	Beam Python
Dataflow JSON Data Producer	`pipelines/json_producer` — TARGET JSON + schema validation → dual sink	Beam Python
Cloud Composer Orchestrator	`composer/dags/mig_000001_1.py`	Airflow 2.10
Reconciliation Services	`apps/recon-service` — source recon + transformation/load recon	Java
Migrability / Reconciliability Reports	JSON + HTML emitted by recon-service	Java
File Storage (JSON out)	second fake-GCS bucket	—
Loader App	`apps/loader-app` — download/consume, push with retries, emit load `.CHS/.ERR/.RPT/.FLG`	Java
Target System	`apps/target-system-mock` — REST stand-in, injects 429/500	Java

Repository layout

```
test-gcp-dataflow-dataform-cloud-composer/
├─ README.md                     Makefile          .env.example
├─ docs/           inputs/ (copies of the 3 source docs + architecture diagram), architecture.md,
runbook-gcp.md
├─ contracts/      ← contract-isolation layer; the ONLY thing a new project edits
│  ├─ tds/         tds-src-project{1,2}.def, tds-target-project{1,2}.def
│  ├─ mappings/    mapping-project{1,2}.yaml
│  ├─ copybooks/   ACCOUNT.cpy, TRANSACTION.cpy
│  └─ schemas/     target-account.schema.json, target-transaction.schema.json
├─ harness/        synthetic Db2 generator (valid/excluded/malformed/duplicate mix)
├─ apps/           Maven multi-module: extractor-app, loader-app, recon-service, target-system-mock
├─ pipelines/      Beam Python: common/, file_processor/, data_enrichment/, json_producer/
├─ dataform/       workflow_settings.yaml, definitions/*.sqlx
├─ composer/dags/  mig_000001_1.py
├─ local/          docker-compose.yml, scripts/
├─ terraform/      envs/dev,
modules/{bootstrap,network,storage,bigquery,dataform,composer,dataflow,iam,kafka,secrets}
└─ tests/          unit + end-to-end acceptance assertions
```

`git init` the directory first (it is not currently a repo). Copy `architecture diagram` into `docs/inputs/` so the repo is self-contained.

Key designs

TDS — dual `.DAT` + JSON (must-prove #2)

Implements the TDS contract. One parser, two field formats, separate SRC and TARGET definitions:

```
record ACCOUNT
  field ACCT_ID     format=DAT   offset=0  len=12
  field CLIENT_TYPE format=DAT   offset=12 len=2
  field META        format=JSON  path=$.metadata
```

`format` defaults to `DAT` (backward compatible). `DAT` fields resolve by fixed offset in copybook mode, by column index in CSV mode — the switch chosen in `mapping-*.yaml`, not code. Reference implementation in `pipelines/common/tds.py`; Java apps read the same files via a small port so there is one contract, two readers.

Two doors + balancing equation (must-prove #1)

Every record leaves through exactly one of **migrated / not migrated**, and a record that was not migrated carries its disposition — **excluded / rejected / deduplicated**. Each run report closes, per run *and* per batch:

```
SRC_read = migrated + not_migrated
         = TARGET_written + excluded_by_filter + rejected + duplicates_dropped
```

Rejects carry source key, batch id, stage (`parse|filter|map|schema`), an **enumerated** reason code (`PARSE_TRUNCATED`, `PARSE_INVALID_JSON`, `MAP_MISSING_REQUIRED`, `SCHEMA_INVALID`, …) and the raw record. Counters are Beam metrics, persisted to a `run_ledger` BQ table so recon reads them rather than re-deriving. **A run that does not balance fails the DAG.**

Run id / window + delta (must-prove #4)

Every artefact, BQ row and report is stamped `run_id`, `window_from`, `window_to`. The Composer DAG is parameterised on them; recon always scopes to one `run_id`. Demo = initial load, then one delta run, with the delta's recon proving keys outside the window were untouched.

Zero-diff `project2` (must-prove #3)

`project2` ships as *new files only* — `tds-src-project2.def`, `tds-target-project2.def`, `mapping-project2.yaml` — with different field names, a different filter predicate and one transform `project1` never uses. Enforced mechanically by a test that runs `git diff --exit-code` over `pipelines/`, `apps/` and `dataform/definitions/` across the two runs.

Sink adapter (both outputs)

`pipelines/common/sinks.py` defines one `TargetWriter` port with two implementations, both driven from config:

- **Kafka** — 200-element batches; the batching contract is resolved as *one message per record, 200 per produce request*, isolated in one class.
- **GCS JSON** — files written for the Loader App to download, per `architecture diagram`.

The same port pattern covers BigQuery: `NativeBigQueryWriter` (real GCP) vs `EmulatorBigQueryWriter` (batched `insertAll`, local) — because the emulator does not support load jobs. *Status: the seam exists and is selected on `bq_is_emulator`, but both bodies currently call `insertAll`; `beam.io.WriteToBigQuery` with FILE_LOADS is the planned real-GCP implementation and is not written yet.*

Local stack (`local/docker-compose.yml`)

Service	Image	Stands in for
`fake-gcs`	`fsouza/fake-gcs-server`	GCS (landing + json-out buckets)
`bq`	`ghcr.io/goccy/bigquery-emulator`	BigQuery
`airflow`	`apache/airflow:2.10.x-python3.11`	Cloud Composer
`redpanda`	`redpandadata/redpanda`	Kafka (lighter than full Kafka, same API)
`target-system-mock`	built from `apps/target-system-mock`	Target System

Beam runs on **DirectRunner** locally. PGP uses the host `gpg 2.4.4` with a throwaway keypair generated into `local/keys/` by `make bootstrap`.

Environment notes. Host Python is 3.14.6 — too new for Beam and Airflow; `uv` pins a **3.11** venv for `pipelines/` and `harness/`. Docker is **podman 4.9.3** shimming the docker CLI, with `docker-compose v5.2.0` present; `make up` must verify the podman socket is live and fall back to `podman-compose`.

Dataform risk + mitigation. Dataform CLI cannot point at the BigQuery emulator (no `apiEndpoint` in `.df-credentials.json`). Mitigation: models are written as **real** `.sqlx` and compiled with `dataform compile --json`, which is a purely local operation; a small runner (`local/scripts/run_dataform.py`) executes the compiled statements in dependency order against whichever BQ endpoint is configured. This keeps the artefacts production-portable. Escape hatch: `BQ_TARGET=real` runs the same SQLX against the free BigQuery sandbox project (`<bq-sandbox-project>`) — zero cost, no billing needed, and it validates any SQL the emulator cannot parse.

Terraform — real GCP infrastructure

`terraform/modules/`, composed by `terraform/envs/dev`:

Module	Provisions
`bootstrap`	project, billing link, ~12 service APIs, TF state bucket
`network`	VPC, subnet, Private Google Access, Cloud NAT (Dataflow workers need egress)
`storage`	buckets: landing, staging, json-out, recon, dataflow-temp + lifecycle rules
`bigquery`	`bq_extraction`, `bq_transformation`, `bq_recon` datasets + table schemas
`dataform`	Dataform repository, release config, workflow config
`composer`	Cloud Composer 2 environment, env vars, DAG bucket sync
`dataflow`	Artifact Registry repo, Flex Template specs in GCS
`iam`	per-component service accounts, least-privilege bindings
`kafka`	Managed Service for Apache Kafka cluster + topic
`secrets`	Secret Manager entries for the PGP private key and Target System credentials

Terraform cannot bootstrap itself from nothing — `docs/runbook-gcp.md` documents the manual prerequisites: reopen a billing account, `gcloud projects create`, link billing, grant the TF service account, then `terraform init/plan/apply`. Until billing is reopened this is `terraform validate`-able but not appliable, which is the honest state and will be stated as such in the README.

> **Cost warning to surface in the runbook:** Cloud Composer 2 alone runs ≈ $300-400/month even idle. `envs/dev` will default Composer to `count = 0` behind a `enable_composer` flag so the rest of the stack can be applied cheaply.

Infrastructure installation steps

Both paths ship as `make` targets and are documented in `docs/runbook-gcp.md`.

A. Local stack — works today, no billing needed

```
        git init
        make bootstrap        # uv pins Python 3.11, generates a throwaway PGP keypair into
local/keys/, writes .env
        make up               # compose: fake-gcs, bigquery-emulator, redpanda, airflow, target-
system-mock
        make init-infra       # creates GCS buckets, BQ datasets + tables, Kafka topic, Airflow
connections
        make verify-stack     # healthchecks every service, fails loudly if podman networking is
broken
```

B. Real GCP

Terraform cannot create its own project or state bucket from nothing, so there is a manual prologue:

```
        # 1. Reopen a billing account in the console, then confirm:
        gcloud auth login && gcloud auth application-default login
        gcloud billing accounts list            # must show OPEN: True

        export TF_VAR_billing_account=XXXXXX-XXXXXX-XXXXXX
        export TF_VAR_project_id=mig-000001-1-dev
        export TF_VAR_region=europe-west1

        # 2. One-time bootstrap: project, APIs, TF state bucket — with local state
        cd terraform/envs/dev
        terraform init -backend=false
        terraform apply -target=module.bootstrap

        # 3. Move state into the bucket just created, then apply the rest
        terraform init -migrate-state
        terraform plan -out=tfplan
        terraform apply tfplan                  # add -var=enable_composer=true when you accept the
~$350/mo
```

Post-apply, the artefacts Terraform deliberately does not own:

```
        make build-templates   # docker build + push the 3 Dataflow Flex Templates to Artifact
Registry
        make deploy-dags       # gsutil rsync composer/dags -> the Composer DAG bucket
        make deploy-dataform   # push dataform/ to the linked git remote, create the release config
        make seed-secrets      # PGP private key + Target System credentials into Secret Manager
        make smoke-gcp         # one tiny run end-to-end on real infrastructure
```

Teardown is `terraform destroy`; buckets have `force_destroy` set only in `envs/dev`.

Local → real GCP: what actually changes

Beyond reopening billing and creating a project. The whole point of the adapter layers above is to keep this list short and mechanical — but it is **not** just flipping a flag, and the runbook will say so.

What does NOT change (the payoff for the adapter design)

`.sqlx` model bodies · TDS definitions · `mapping-*.yaml` · JSON Schemas · the two-door and balancing logic · Beam `DoFn` transform logic · Java app business logic. If any of these need editing at cutover, an adapter boundary was drawn in the wrong place.

Code and configuration changes

#	Area	Local	Real GCP
1	**Beam runner**	`DirectRunner`	`DataflowRunner` + `--region`, `--temp_location`, `--staging_location`, `--service_account_email`, `--subnetwork`, `--max_num_workers`, `--no_use_public_ips`
2	**Pipeline packaging**	run from source	**Flex Templates** — Docker image → Artifact Registry, template spec JSON → GCS. Composer can only launch templates, so this is required work, not a flag
3	**BigQuery sink**	`EmulatorBigQueryWriter` (batched `insertAll`)	`NativeBigQueryWriter` — *currently also* `insertAll`; target is `WriteToBigQuery` / `FILE_LOADS` / `custom_gcs_temp_location`
4	**BQ table design**	flat tables	**partitioning + clustering mandatory** — at 20B rows an unpartitioned table is both unqueryable and a cost incident
5	**Object storage**	`STORAGE_EMULATOR_HOST` → fake-gcs	unset the var, real `gs://` URIs, ADC credentials. Same change in the Java apps
6	**Dataform**	`dataform compile --json` + local runner shim	real **Dataform repository** linked to a Git remote, with release + workflow configs; Composer invokes it via `DataformCreateCompilationResultOperator` + `DataformCreateWorkflowInvocationOperator`. The SQLX files are unchanged — that is why they are written as real SQLX
7	**Orchestration**	Airflow container, local callables	Composer 2. DAG logic survives; **operators change** to `DataflowStartFlexTemplateOperator`, `DataformCreateWorkflowInvocationOperator`, `BigQueryInsertJobOperator`, `GCSObjectExistenceSensor` for the `.FLG` semaphore. DAGs deploy by syncing to the Composer DAG bucket; PyPI deps move to the environment config
8	**Kafka**	Redpanda, `PLAINTEXT`	Managed Service for Apache Kafka — new bootstrap servers **and** SASL/OAUTHBEARER auth against a service account. A real client-config change, not a URL swap
9	**Identity**	no auth	one least-privilege service account per component; Workload Identity for Composer; ADC everywhere. Route every client construction through one factory so this is a single edit
10	**Secrets**	`local/keys/` PGP pair, plaintext env	Secret Manager: PGP private key and Target System credentials fetched at runtime, never on worker disk
11	**Target System**	`target-system-mock`	real endpoint — needs a network path (Private Service Connect / VPN), real TLS or OAuth credentials, and real rate limits. Owned by the other team, so it stays behind the loader adapter
12	**Networking**	none	VPC + subnet with **Private Google Access**, Cloud NAT for worker egress, firewall rule `tcp:12345-12346` between Dataflow workers. Workers must run without public IPs

Production concerns that only exist in GCP

- **Quotas** — Dataflow CPU-per-region, BQ load jobs (1,500/table/day, a real ceiling at 100M batches), Storage Write API limits. Request increases *before* the first large run.
- **Monitoring** — Cloud Monitoring dashboards, log-based metrics on reject counts, and an alert when the balancing equation fails. Locally `make verify` catches this; in GCP nobody is watching the terminal.

- **CMEK, VPC Service Controls, data residency** — table stakes for a core-banking migration, absent locally.
- **Data masking at ingest** — the moment real banking data enters any non-prod environment, with a stable pseudonymisation mapping so reconciliation still works.
- **Cost controls** — Dataflow autoscaling caps, Composer sizing, BQ on-demand vs reservation.
- **CI/CD** — build and push Flex Template images, sync DAGs, `terraform apply` per environment.

Honest estimate: items 1, 3, 5, 9 are genuinely config flips if the adapters hold. Items **2, 6, 7, 8, 12** are real implementation work — roughly a phase of their own — and item 11 is blocked on the other team regardless.

Build sequence

Each phase ends green and runnable before the next starts.

#	Phase	Output
0	Repo skeleton, `git init`, Makefile, contracts dir, `architecture diagram` copied to `docs/inputs/`, **this plan written to `docs/PLAN.md`**	`make help` works, `docs/PLAN.md` present
1	Harness + TDS parser + `contracts/` for `project1`	Unit tests green on all four record classes
2	Lane E: extractor-app → 5 artefacts → archive → gzip → PGP → fake-GCS	`.FLG` lands in the landing bucket
3	Local stack up: fake-gcs, bq emulator, redpanda, airflow, Target System mock	`make up` + healthchecks pass
4	Pipeline 1 — file_processor: decrypt/decompress, `.CHS` verify, TDS parse, parse/filter/dedup → BQ Extraction	Balancing equation closes on a seeded run
5	Dataform SQLX: Extraction → Transformation dataset	`dataform compile` clean; rows land in `bq_transformation`
6	Pipeline 2 — data_enrichment	Enriched columns populated
7	Pipeline 3 — json_producer: TARGET JSON + JSON-Schema validation → **both** sinks	200-element Kafka batches + JSON files in GCS
8	Lane L: loader-app + target-system-mock, retries/backoff/idempotency, load-side `.CHS/.ERR/.RPT/.FLG`	Injected 429/500 recovered, artefacts written back
9	recon-service: source recon, transformation/load recon, migrability + reconciliability reports	Reports JSON + HTML, equation closes
10	Composer DAG wiring all of the above, parameterised on `run_id`/window	One-command end-to-end
11	Delta run — initial + one delta, scoped correctly	Must-prove #4
12	`project2` — new YAML + TDS only	`git diff --exit-code` proves must-prove #3
13	Terraform modules + `runbook-gcp.md`	`terraform validate` + `plan` clean

Verification

End-to-end, one command:

```
make up && make run-initial && make verify
```

`make verify` asserts, and fails loudly on any of:

1. Balancing equation closes exactly — per run **and** per batch
2. Seeded excluded count matches the harness's known count **exactly**
3. Every seeded malformed record appears in `.ERR` with the correct enumerated reason code
4. 100% of emitted TARGET JSON validates against the TARGET JSON Schema
5. Kafka batch count == `ceil(records / 200)`, verified on the topic
6. Every non-excluded, non-rejected SRC key appears exactly once in TARGET, scoped to `run_id`
7. All five artefact types present in both the extraction and load lanes, `.FLG` last
8. `.CHS` checksums verify on both sides

Then:

```
make run-delta      # initial + one delta, recon scoped by run_id/window   → must-prove #4
make verify-project2  # runs mapping-project2, then git diff --exit-code   → must-prove #3
make tf-validate      # terraform fmt -check && validate on envs/dev
```

Unit tests: `uv run pytest` for pipelines/harness, `mvn test` for the Java apps.

Risks

Risk	Mitigation
BigQuery emulator rejects Dataform-generated SQL	Keep SQLX to a supported subset; `BQ_TARGET=real` runs it against the free BQ sandbox
Beam `WriteToBigQuery` needs load jobs the emulator lacks	Sink adapter — `EmulatorBigQuerySink` uses batched `insertAll`
Podman-as-docker breaks compose networking	Verified in phase 3 before anything depends on it; `podman-compose` fallback
Host Python 3.14 unsupported by Beam/Airflow	`uv` pins 3.11 for pipelines; Airflow runs containerised
Terraform not appliable while billing is closed	Ship `validate`-clean code + runbook; state the limitation plainly in the README
Scope is large	Phases 0-10 give a working end-to-end slice; 11-13 add the remaining proofs

Out of scope

Field2Field reconciliation, ThoughtWorks import-results recon (contract undefined), real Db2 connectivity, the 8,000 rec/s throughput benchmark (a separate exercise once the slice runs), and production hardening (CMEK, VPC-SC, real PGP key management).